

'return-to-libc' being the most common approach. Yet another approach is to exploit a bug in kernel. Since the kernel of an OS has all the privileges possible, a security bug in kernel would allow the attacker to execute just about anything on the compromised system. The most prominent method to do this is by injecting the code in the application's user memory and exploiting a kernel security bug to execute the injected code. This is a deadly method of attack since the kernel has virtually no restrictions on what it can do. Exploiting a kernel bug means exploiting the privileges available with the process which makes the system extremely vulnerable to damage.

This is especially true for versions before Windows 8 and Intel processors released prior to the advent of the Ivy Bridge architecture. Ivy Bridge architecture brings in a security mechanism called SMEP ('Supervisor Mode Execution Prevention' or 'Supervisory Mode Execute Protection'). SMEP disables the kernel from executing any code in the user space of the process. This enables protection against attacks on the kernel itself. With SMEP deployed, the kernel can't be instructed to execute any code residing in the user area of the process (it's where most attackers would save the injected code) and since the process itself doesn't get the permission to alter the kernel level memory, the injected code in user code is basically useless.

With the user-memory locked out, one of the very few options left for an attacker is to somehow place the exploit (injected code) in the kernel memory. Windows 8 elevates its security again; since on most occasions the malicious code is injected by making it look like data, the code gets stored in the 'data' section. Traditionally, all of the memory owned by the kernel was executable; one could command the kernel to execute data and it would do it. In Windows 8 however, even the code located in kernel memory is executed only if it's marked as executable by the DEP faculty. This prevents cases where the injected code is passed to kernel via an API call.

AppContainers and Sandboxing

The Modern UI apps of Windows 8 – all running in sandboxes – are allowed to access only the bare necessities and can't access beyond the permitted limits. They read and write to only their respective private storage areas.

AppContainers is a new Integrity Level which has bare minimum permissions. If the app wants to perform an action, it must be specified in a manifest file which contains the list of capabilities required by the application. If the application doesn't specify the permission to access the resource